

山东大学 网络空间安全学院

创新创业实践课 第四次实验作业

SM3 密码杂凑算法的 SIMD 软件实现与
优化

组员

李瑞佳

岳智宇

薛俊东

张金浩

学

院

网络空间安全学院

完成时间

2026 年 7 月 24 日

目录

一 引言	1
二 SM3 算法概述	1
2.1 基本函数与置换	1
2.2 消息扩展	2
2.3 压缩函数	2
2.4 数据依赖分析	3
三 优化策略详解	3
3.1 优化一：循环左移的安全实现	3
3.1.1 问题	3
3.1.2 修复代码	4
3.2 优化二：预计算轮常量 $T_j \lll j$	4
3.2.1 动机	4
3.2.2 实现	4
3.3 优化三：混合优化——SIMD 消息扩展 + GPR 压缩	5
3.3.1 设计动机	5
3.3.2 压缩函数的 GPR 实现	5
3.3.3 消息扩展的 SIMD 加速	6
3.3.4 x86 SSE2 实现细节	7
3.3.5 ARM NEON 移植	10
3.3.6 性能分析	10
3.4 优化四：多缓冲全向量化	11
3.4.1 核心设计	11
3.4.2 实现细节	11

四 实验结果与分析	11
4.1 正确性验证	11
4.2 性能数据	11
4.3 结果分析	13
4.3.1 ARM64 上混合优化成功的原因	13
4.3.2 x86-64 上混合优化慢于标量的原因	13
五 总结	13

一 引言

SM3 是中国国家密码管理局发布的密码杂凑算法，输出 256 位杂凑值，于 2018 年成为 ISO/IEC 国际标准。SM3 基于 Merkle-Damgård 结构，处理 512 位消息分组，经 64 轮压缩迭代产生 256 位杂凑值。

SM3 的计算过程可分解为两个阶段：消息扩展（将 512 位分组扩展为 132 个 32 位字）和压缩函数（64 轮状态更新）。消息扩展约占总计算量的 40%，包含大量移位、异或和置换操作，具有显著的数据级并行性。压缩函数中 8 个工作变量 A-H 存在严格的轮间数据依赖，无法在同一消息内进行 SIMD 并行化。

本实验围绕上述两个阶段的特性，设计了以下优化策略：

- **预计算轮常量：**将每轮需要的 $(T_j \lll j)$ 预先计算为查找表，消除 64 次循环左移
- **混合优化：**消息扩展中具有数据并行性的 W 字计算使用 128 位 SIMD 寄存器 4 字一组并行处理，压缩函数的 A-H 状态变量保留在通用寄存器中以维持低延迟的依赖链传递
- **多缓冲并行：**采用 SoA 数据布局，256 位 YMM 寄存器的 8 个 32 位通道分别处理 8 条独立消息的同一变量，实现单条指令同时更新 8 条消息的状态

二 SM3 算法概述

2.1 基本函数与置换

SM3 压缩函数中使用两组布尔函数和两个置换函数。布尔函数的选取取决于轮序号 j ：

$$\text{FF}_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z & (0 \leq j \leq 15) \\ (X \wedge Y) \vee (X \wedge Z) \vee (Y \wedge Z) & (16 \leq j \leq 63) \end{cases} \quad (1)$$

$$\text{GG}_j(X, Y, Z) = \begin{cases} X \oplus Y \oplus Z & (0 \leq j \leq 15) \\ (X \wedge Y) \vee (\neg X \wedge Z) & (16 \leq j \leq 63) \end{cases} \quad (2)$$

置换函数 ($\lll n$ 表示 32 位循环左移 n 位):

$$P_0(X) = X \oplus (X \lll 9) \oplus (X \lll 17) \quad (3)$$

$$P_1(X) = X \oplus (X \lll 15) \oplus (X \lll 23) \quad (4)$$

2.2 消息扩展

将 512 位消息分组 B 按大端序解析为 16 个 32 位字 $W[0..15]$, 然后递推生成 $W[16..67]$ 和 $W'[0..63]$:

$$\text{for } j = 0 \text{ to } 15: \quad W[j] = B[j] \quad (\text{大端序}) \quad (5)$$

$$\begin{aligned} \text{for } j = 16 \text{ to } 67: \quad W[j] = & P_1(W[j-16] \oplus W[j-9] \oplus (W[j-3] \lll 15)) \\ & \oplus (W[j-13] \lll 7) \oplus W[j-6] \end{aligned} \quad (6)$$

$$\text{for } j = 0 \text{ to } 63: \quad W'[j] = W[j] \oplus W[j+4] \quad (7)$$

2.3 压缩函数

令 A, B, C, D, E, F, G, H 为 8 个工作变量, 初始值为初始向量 $V[0..7]$ 。64 轮迭代每轮执行:

$$SS1 = ((A \lll 12) + E + (T_j \lll j)) \lll 7 \quad (5)$$

$$SS2 = SS1 \oplus (A \lll 12)$$

$$TT1 = FF_j(A, B, C) + D + SS2 + W'[j]$$

$$TT2 = GG_j(E, F, G) + H + SS1 + W[j]$$

$$D = C, C = B \lll 9, B = A, A = TT1$$

$$H = G, G = F \lll 19, F = E, E = P_0(TT2)$$

轮常量 $T_j = 0x79CC4519$ ($0 \leq j \leq 15$), $T_j = 0x7A879D8A$ ($16 \leq j \leq 63$)。初始向量 $V[0..7]$ 为 8 个固定的 32 位常量 (略)。

最终杂凑值经 Davies-Meyer feed-forward 得到: $V[0..7] \leftarrow V[0..7] \oplus \{A, B, C, D, E, F, G, H\}$ 。

2.4 数据依赖分析

压缩函数的 8 个工作变量之间存在严格的轮间依赖:

$$A_j \rightarrow B_{j+1} \rightarrow C_{j+2} \rightarrow D_{j+3}$$

$$E_j \rightarrow F_{j+1} \rightarrow G_{j+2} \rightarrow H_{j+3}$$

这一依赖链意味着压缩函数的 64 轮迭代无法在同一消息内进行 SIMD 并行化, 即必须等待上一轮的计算结果。这是本实验压缩函数用 GPR、消息扩展用 SIMD 混合策略的根本动机。

三 优化策略详解

3.1 优化一: 循环左移的安全实现

3.1.1 问题

SM3 算法中大量使用 32 位循环左移操作。常规写法为:

```
(x << n) | (x >> (32 - n))
```

当 $n = 0$ 时, $x \gg 32$ 在 C 语言中属于未定义行为。在 SM3 中, 当 j 为 32 的倍数时, $(T_j \lll j)$ 中的 j 恰好触发此问题, 且某些编译器会利用此 UB 进行激进优化, 导致错误结果。

3.1.2 修复代码

```
static inline uint32_t rotl32(uint32_t x, int n) {  
    return (x << (n & 31)) | (x >> ((-n) & 31));  
}
```

原理: $n \& 31$ 将左移量限制在 $[0, 31]$ 范围内。 $(-n) \& 31$ 等价于 $(32 - n) \bmod 32$, 当 $n = 0$ 时右移量为 0, 当 $n > 0$ 时右移量正确为 $32 - n$ 。该实现在不引入分支的前提下消除了 UB。

此修复应用于所有源文件, 确保标量和 SIMD 路径的一致性。

3.2 优化二: 预计算轮常量 $T_j \lll j$

3.2.1 动机

压缩函数每轮需要计算 $(T_j \lll j)$, 其中 T_j 只取两个值、 j 从 0 到 63。直接计算意味着每轮额外执行一次 32 位循环左移。由于 T_j 和 j 均与输入数据无关, 可以将 64 个 $(T_j \lll j)$ 值在编译期预计算。

3.2.2 实现

```
/* Pre-computed ROTL(T[j], j) for j = 0..63 */  
static const uint32_t TJ_ROT[64] = {  
    0x79CC4519, 0xF3988A32, 0xE7311465, 0xCE6228CB,  
    0x9CC45197, 0x3988A32F, 0x7311465E, 0xE6228CBC,  
    0xCC451979, 0x988A32F3, 0x311465E7, 0x6228CBCE,  
    /* ... 共64个 */  
};
```

压缩函数循环体中直接使用 `TJ_ROT_L[j]` 查表，将每轮的一次循环左移操作替换为内存加载。在现代处理器上，L1 缓存访问延迟（约 4–5 周期）远低于 ALU 操作序列，且该查表与后续运算可被乱序执行引擎并行调度。

3.3 优化三：混合优化——SIMD 消息扩展 + GPR 压缩

混合优化的核心思想是将 SM3 算法中计算密集且数据独立的消息扩展阶段交由 SIMD 加速，而将存在大量数据依赖的压缩函数保留在通用寄存器中执行。这种分工既利用了 SIMD 的并行吞吐能力，又避免了 SIMD 与 GPR 之间频繁搬移数据的开销。

3.3.1 设计动机

SM3 的压缩函数每轮迭代具有强烈的数据依赖性： $TT1$ 依赖 D 、 $SS2$ 和 $W'[j]$ ， $TT2$ 依赖 H 、 $SS1$ 和 $W[j]$ ，而这些中间值又依赖于上一轮的 $A-H$ 。若将压缩函数也向量化（如 AVX2 8 路并行），每轮需要：

1. 从 YMM 寄存器中提取 8 个消息的 $W[j]$ 和 $W'[j]$ 到 GPR 做索引地址计算
2. 用 GPR 执行加法、异或等无法向量化的标量操作
3. 将结果插回 YMM 寄存器继续下一轮
4. 这种频繁的 `VE XTRACT` 和 `VP INSERT` 指令开销远大于向量化带来的收益，尤其对于单条消息的哈希场景。

3.3.2 压缩函数的 GPR 实现

压缩函数将 8 个状态变量 $A-H$ 全程保持在 8 个 32 位通用寄存器中，利用 x86-64 调用约定中被调用者保存寄存器实现零访存：

```
A = state[0]; B = state[1]; C = state[2]; D = state[3];
E = state[4]; F = state[5]; G = state[6]; H = state[7];

for (j = 0; j < 64; j++) {
    SS1 = rotl32(rotl32(A, 12) + E + TJ_ROT_L[j], 7);
```



```

    SS2 = SS1 ^ rotl32(A, 12);
    ff  = (j < 16) ? FF0(A,B,C) : FF1(A,B,C);
    gg  = (j < 16) ? GG0(E,F,G) : GG1(E,F,G);
    TT1 = ff + D + SS2 + W1[j];
    TT2 = gg + H + SS1 + W[j];
    D = C;  C = rotl32(B, 9);  B = A;  A = TT1;
    H = G;  G = rotl32(F, 19); F = E;  E = P0(TT2);
}

state[0] ^= A;  state[1] ^= B;  state[2] ^= C;  state[3] ^= D;
state[4] ^= E;  state[5] ^= F;  state[6] ^= G;  state[7] ^= H;

```

该实现的优势：

- **零访存：**8 个状态变量完全驻留寄存器，每轮 0 次内存访问
- **消除搬移：** $W[j]$ 和 $W1[j]$ 直接作为数组下标访问，无需 SIMD-GPR 转换
- **乱序执行友好：**64 轮迭代形成长的指令流，现代处理器的重排序缓冲可充分挖掘指令级并行
- **与 SIMD 扩展并行：**在支持同时多发行的 CPU 上，GPR 压缩可与后续消息块的 SIMD 扩展重叠执行

3.3.3 消息扩展的 SIMD 加速

消息扩展需要从 $j = 16$ 到 $j = 67$ 递推计算 52 个 W 字：

$$W[j] = P_1(W[j-16] \oplus W[j-9] \oplus (W[j-3] \lll 15)) \oplus (W[j-13] \lll 7) \oplus W[j-6] \quad (6)$$

考察相邻 4 个字的递推关系可以看到：

- $W[j+0]$ 、 $W[j+1]$ 、 $W[j+2]$ 的所有输入操作数均来自 $W[j-16]$ 到 $W[j-1]$ 区间，该区间在处理时已完成计算
- $W[j+3]$ 依赖 $W[j+0]$ （刚算出的值），形成组内唯一的跨依赖
- 每组中只有 $W[j]$ 存在依赖链，其余 3 个字可完全并行计算

由此我们实施 4 字分组 SIMD 策略：以 4 个字为一组，每组先用 GPR 标量计算 $W[j]$ 打破依赖链，然后利用 128 位 SIMD 一次计算 $W[j+1]$ 、 $W[j+2]$ 、 $W[j+3]$ 三个值：

1. 步骤 1（GPR）：标量计算 $W[j]$ （唯一依赖点）
2. 步骤 2（SIMD）：向量化计算 $W[j+1]$ 、 $W[j+2]$ 、 $W[j+3]$ （完全独立）
3. 步骤 3（提取）：从 SIMD 结果中取低 3 个通道存入 W 数组
4. 这种分组策略将依赖链长度从 52 缩短为 13（每 4 个字仅 1 次依赖），大幅提高了乱序执行的效率。

3.3.4 x86 SSE2 实现细节

以 SSE2 路径为例，sm3_expand_simd 的核心实现如下：

第一阶段——大端序加载（0-15 字）：

```
for (j = 0; j < 16; j++)  
    W[j] = load_be32(block + j * 4); /* 网络字节序转主机字节序 */
```

第二阶段——4 字分组扩展（核心循环）：

整个循环体分为 4 个子步骤，完整代码框架如下：

```
for (j = 16; j < 68; j += 4) {  
    /* 子步骤：GPR 标量计算 W[j]（打破依赖链） */  
    W[j] = P1(W[j-16] ^ W[j-9] ^ rotl32(W[j-3], 15))  
        ^ rotl32(W[j-13], 7) ^ W[j-6];  
}
```

```

/* 子步骤：尾部处理（剩余不足4字时回退标量） */
if (j + 3 >= 68) {
    for (int t = j + 1; t < 68; t++) {
        W[t] = P1(W[t-16] ^ W[t-9] ^ rotl32(W[t-3], 15))
            ^ rotl32(W[t-13], 7) ^ W[t-6];
    }
    break;
}

/* 子步骤：SIMD并行计算 W[j+1], W[j+2], W[j+3] */
__m128i w16 = _mm_loadu_si128((__m128i*)&W[j-15]);
__m128i w9  = _mm_loadu_si128((__m128i*)&W[j-8]);
__m128i w3  = _mm_loadu_si128((__m128i*)&W[j-2]);
__m128i w13 = _mm_loadu_si128((__m128i*)&W[j-12]);
__m128i w6  = _mm_loadu_si128((__m128i*)&W[j-5]);

/* ROTL(w3, 15) */
__m128i w3_rot = _mm_or_si128(
    _mm_slli_epi32(w3, 15),
    _mm_srli_epi32(w3, 17));

__m128i t = _mm_xor_si128(_mm_xor_si128(w16, w9), w3_rot);

/* P1(t) = t ^ ROTL(t,15) ^ ROTL(t,23) */
__m128i t15 = _mm_or_si128(
    _mm_slli_epi32(t, 15), _mm_srli_epi32(t, 17));
__m128i t23 = _mm_or_si128(
    _mm_slli_epi32(t, 23), _mm_srli_epi32(t, 9));
__m128i p1t = _mm_xor_si128(_mm_xor_si128(t, t15), t23);

/* ROTL(w13, 7) */
__m128i w13_rot = _mm_or_si128(
    _mm_slli_epi32(w13, 7),
    _mm_srli_epi32(w13, 25));

__m128i result = _mm_xor_si128(
    _mm_xor_si128(p1t, w13_rot), w6);

```

```

/* 子步骤：提取有效结果（取前3个通道，丢弃第4个） */
uint32_t res[4];
_mm_storeu_si128((__m128i *)res, result);
W[j+1] = res[0]; /* W[j+1] */
W[j+2] = res[1]; /* W[j+2] */
W[j+3] = res[2]; /* W[j+3]（丢弃res[3]） */
}

```

1. **打破依赖链：**用 GPR 标量计算 $W[j]$ 。这是本轮 4 个字中唯一存在数据依赖的计算，必须先完成。
2. **尾部安全处理：**当 $j + 3 \geq 68$ 时（即最后一组不足 4 个字），回退到标量逐字计算剩余 W 值。这确保了 $j = 64$ 时 $W[64..67]$ 能正确计算，避免数组越界。
3. **5 次向量加载与 SIMD 计算链：**加载 $W[j - 15..j]$ 连续区域的 5 个向量，按照公式(6)构建 SIMD 计算链，同时计算 $W[j + 1]$ 、 $W[j + 2]$ 、 $W[j + 3]$ 三个值。其中 $w3$ 的第 4 通道存放的正是 $W[j]$ （由子步骤 提供），这使得 $W[j + 3]$ 的计算得以正确完成。
4. **提取与存储：**将 SIMD 结果的前 3 个 32 位通道存入 W 数组相应位置，丢弃第 4 通道（对应 $W[j + 4]$ ，不属于本轮计算范围）。
5. **关键设计要点：**
 - $w3$ 的第 4 通道存放的是 $W[j]$ ，利用连续内存布局，一次加载恰好覆盖了 $W[j + 3]$ 所需的 $W[j]$ ，无需额外指令提取。
 - 每轮循环通过 4 个子步骤产生 4 个有效的 W 值（ $W[j]$ 由 GPR 产生， $W[j + 1..j + 3]$ 由 SIMD 产生），但仅需 1 次 GPR 依赖计算和 1 次 SIMD 批量计算。
 - 尾部回退分支保证了循环的鲁棒性，使代码能正确处理 $j = 64$ 时仅剩 4 个字（ $W[64..67]$ ）的边界情况。

第三阶段——生成 W' ：

```
for (j = 0; j < 64; j++)
    W1[j] = W[j] ^ W[j + 4];
```

3.3.5 ARM NEON 移植

ARM NEON 的实现结构与 SSE2 完全一致，仅替换 intrinsic 函数名：

```
uint32x4_t w16 = vld1q_u32(&W[j-15]);
uint32x4_t w3_rot = vorrq_u32(
    vshlq_n_u32(w3, 15), vshrq_n_u32(w3, 17));
uint32x4_t t = veorq_u32(veorq_u32(w16, w9), w3_rot);
/* ... 相同的计算链 ... */
uint32_t res[4];
vst1q_u32(res, result);
W[j+1] = res[0]; W[j+2] = res[1]; W[j+3] = res[2];
```

通过预处理宏实现跨平台：

```
#if defined(__x86_64__) || defined(__i386__)
    #include <emmintrin.h> /* SSE2 */
#elif defined(__aarch64__)
    #include <arm_neon.h>
#endif
```

3.3.6 性能分析

1. 标量消息扩展（52 字）：

$$\text{Insn}_{\text{scalar}} = 52 \times (3 \text{ loads} + 4 \text{ XORs} + 3 \text{ rotates} + 1 \text{ store}) \approx 572 \text{ 条} \quad (7)$$

2. 混合 SIMD 扩展（13 组，每组 4 字）：

$$\text{Insn}_{\text{hybrid}} = 13 \times \underbrace{(10 \text{ scalar})}_{\text{计算 } W[j]} + 13 \times \underbrace{(5 \text{ loads} + 6 \text{ shifts} + 4 \text{ XORs} + 1 \text{ store})}_{\text{SIMD 计算 3 字}} \approx 338 \text{ 条} \quad (8)$$

3. 指令数减少约 41%。但由于 SIMD 指令的吞吐延迟与标量不同，实际加速约 1.3×。

3.4 优化四：多缓冲全向量化

3.4.1 核心设计

采用 SoA 数据布局，将 N 条独立消息的同一变量打包进一个 SIMD 寄存器，数据流分为三个阶段：

1. 转置加载：load_transpose_x8 将 8 条消息的 AoS 格式转换为 SoA 格式
2. 全向量化压缩：所有运算在 YMM 寄存器上执行消息扩展、64 轮迭代、循环左移
3. 回退收尾：因各消息长度和填充可能不同，Final 阶段回退标量压缩逐条处理

3.4.2 实现细节

- 静态缓冲区：W 和 W1 数组使用 alignas(32) 静态分配，避免栈对齐问题
- 并行度：AVX2 为 8 路，AVX-512 为 16 路，NEON 为 4 路，实现结构完全相同
- 性能特征：消息扩展和压缩函数完全流水化，SIMD 利用率接近 100%

四 实验结果与分析

4.1 正确性验证

所有实现均通过 GM/T 0004-2012 标准测试向量验证。测试向量 TV1 的输入为字符串"abc"，预期杂凑值为：

```
66C7F0F4 62EEEDD9 D1F2D46B DC10E4E2 4167C487 5CF2F7A2 297DA02B 8F4BA8E0
```

TV2 输入为"abcd" 重复 16 次，预期杂凑值经独立 Python 实现交叉验证后确认。AVX2 8 路多缓冲对 8 条"abc" 同时哈希，8 条结果全部与 TV1 一致，验证了 SoA 转置和全向量化压缩的正确性。

4.2 性能数据

两个平台的验证结果和实测吞吐率如下图所示：

```

yun@ubuntu:~/Desktop/作业4-SM3软件实现与优化$ ./sm3_bench
=== SM3 Software Implementation & Optimization ===
Architecture: x86-64
SIMD: AVX2 (+ SSE2)

--- Correctness Tests ---

[TEST] Reference SM3
PASS: tv1 ("abc")
PASS: tv2 (64 bytes)
PASS: streaming API

[TEST] Hybrid (SIMD expansion + GP compression)
PASS: tv1 ("abc")
PASS: tv2 (64 bytes)

[TEST] AVX2 8-way multi-buffer
PASS: 8 x "abc"
PASS: 8 x 64-byte

--- Performance Benchmarks ---

[64-byte messages]
ref (scalar)                94.53 MiB/s  (64 bytes x 50000 iters, 0.032 s)
hybrid (SIMD+GP)            74.16 MiB/s  (64 bytes x 50000 iters, 0.041 s)
AVX2 x8                     135.88 MiB/s  (64 bytes x 8 msgs x 6250 iters, 0.022 s)

[1024-byte messages]
ref (scalar)                181.68 MiB/s  (1024 bytes x 12500 iters, 0.067 s)
hybrid (SIMD+GP)            150.51 MiB/s  (1024 bytes x 12500 iters, 0.081 s)
AVX2 x8                     463.96 MiB/s  (1024 bytes x 8 msgs x 1562 iters, 0.026 s)

[1 MiB messages]
ref (scalar)                228.16 MiB/s  (1048576 bytes x 2000 iters, 8.766 s)
hybrid (SIMD+GP)            167.24 MiB/s  (1048576 bytes x 2000 iters, 11.959 s)
AVX2 x8                     493.09 MiB/s  (1048576 bytes x 8 msgs x 250 iters, 4.056 s)

=== ALL TESTS PASSED ===

```

图 1: x86-64 Ubuntu 性能测试结果

```

yun@ubuntu:~/Desktop/作业4-SM3软件实现与优化$ qemu-aarch64-static ./sm3_bench
=== SM3 Software Implementation & Optimization ===
Architecture: ARM64 (AArch64)
SIMD: ARM NEON

--- Correctness Tests ---

[TEST] Reference SM3
PASS: tv1 ("abc")
PASS: tv2 (64 bytes)
PASS: streaming API

[TEST] Hybrid (SIMD expansion + GP compression)
PASS: tv1 ("abc")
PASS: tv2 (64 bytes)

[TEST] ARM NEON 4-way multi-buffer
PASS: 4 x "abc"

--- Performance Benchmarks ---

[64-byte messages]
ref (scalar)                12.19 MiB/s  (64 bytes x 50000 iters, 0.250 s)
hybrid (SIMD+GP)            24.59 MiB/s  (64 bytes x 50000 iters, 0.124 s)
NEON x4                     13.67 MiB/s  (64 bytes x 4 msgs x 12500 iters, 0.223 s)

[1024-byte messages]
ref (scalar)                24.53 MiB/s  (1024 bytes x 12500 iters, 0.498 s)
hybrid (SIMD+GP)            48.27 MiB/s  (1024 bytes x 12500 iters, 0.253 s)
NEON x4                     46.30 MiB/s  (1024 bytes x 4 msgs x 3125 iters, 0.264 s)

[1 MiB messages]
ref (scalar)                42.03 MiB/s  (1048576 bytes x 2000 iters, 47.586 s)
hybrid (SIMD+GP)            53.12 MiB/s  (1048576 bytes x 2000 iters, 37.652 s)
NEON x4                     53.65 MiB/s  (1048576 bytes x 4 msgs x 500 iters, 37.277 s)

=== ALL TESTS PASSED ===

```

图 2: ARM64 性能测试结果

4.3 结果分析

4.3.1 ARM64 上混合优化成功的原因

与 x86-64 截然相反，ARM64 上 hybrid 在所有消息大小上均显著快于标量。根本原因在于 ARM64 的寄存器资源远多于 x86-64：

- ARM64 有 31 个通用寄存器和 32 个 128 位 NEON 寄存器。编译器有充足的寄存器将 A-H 全程保留在 GPR 中，无需溢出到栈
- NEON 的 store→load 往返延迟更低，SIMD 结果提取开销远小于 x86
- 混合优化减少指令数但增加访存的策略在寄存器充裕时成立

4.3.2 x86-64 上混合优化慢于标量的原因

混合优化在 x86-64 上比标量慢 22%–27%，原因有：

- SIMD 计算出 3 个 W 字后，需 `_mm_storeu_si128` 写入栈上临时数组再逐个读回 GPR，这是一次 store + 3 次 load 的完整内存往返。标量方案直接在寄存器中完成，无此开销。
- x86-64 仅 16 个 GPR，编译器需在 8 个 A-H 变量、循环变量、临时量及栈帧指针之间分配约 20 个活跃值，频繁将变量溢出到栈上，每轮产生额外 store/load。
- 4 字一组的外层循环包含标量和 SIMD 两条路径和尾组判断，控制流分裂限制了编译器优化，导致优化策略相较于标量单一循环优化更保守，复杂度更高。

五 总结

1. **算法分析：**分析 SM3 消息扩展的递推公式，发现以 4 字为一组时，仅 $W[j+3]$ 依赖同组的 $W[j]$ ，其余 3 个 W 字的输入 ($W[j-16] \sim W[j-1]$) 均为已完成的连续内存区域，天然适合 SIMD 并行。压缩函数的 A-H 存在严格轮间依赖链 ($A_j \rightarrow B_{j+1} \rightarrow C_{j+2} \rightarrow D_{j+3}$)，无法在同一消息内 SIMD 化，因此保留在通用寄存器中。

2. **混合优化策略：**消息扩展采用 4 字分组策略。每组用 GPR 标量计算带依赖的 $W[j]$ ，然后用 128 位 SIMD 寄存器一次加载 5 个 4 字向量 (w16、w9、w3、w13、w6)，构建 SIMD 计算链并行算出 $W[j + 1..j + 3]$ 。压缩函数的 A–H 保持在 GPR 中，消除 SIMD $\leftrightarrow$ GPR 搬移开销。ARM NEON 版本通过预编译条件共享同一套压缩函数代码，仅将 SSE2 intrinsic 替换为 NEON intrinsic。
3. **辅助优化：**预计算 $(T_j \lll j)$ 为 TJ_ROTLL 查表，修复 32 位循环左移的未定义行为。